

Razvoj web aplikacija - Predavanje 2

Modeliranje podataka, ORM i migracije

mag.ing. Josip Torić (SIT UNIZD)

12.10.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Modeliranje podataka: entiteti, relacije, constrainti
- 2 Normalizacija (minimalno) i tipične greške u modelu
- 3 SQL kao temelj: što ORM zapravo radi
- 4 SQLAlchemy 2.0: modeli, engine, session
- 5 Migracije: Alembic, autogenerate, rollback
- 6 Seed podaci i razvojni workflow

Cilj

Na kraju predavanja imamo bazu s tablicama **Club** i **User**, migracije i seed.

Nakon ovog predavanja student može:

- modelirati tablice i relacije za realnu web aplikaciju
- objasniti razliku: model (DB) vs schema (API DTO)
- postaviti SQLAlchemy engine i session per request
- koristiti Alembic za izradu i primjenu migracija
- objasniti unique/foreign key constraint i zašto su bitni

Prvo: domena našeg projekta (što modeliramo?)

Minimalni entiteti za početak:

- **Club** - klub (naziv, grad, kontakt email, kontakt telefon)
- **User** - korisnik (admin ili club user)

Kasnije dodajemo:

- **Lifter** (natjecatelj), **Competition**, **Registration**

Iterativno: prvo temelj (auth + ownership), tek onda složenije entitete.

Entitet, tablica, redak (terminologija)

- **Entitet** (concept) → **tablica** (implementation)
- **Instanca entiteta** → **redak** (row)
- **Atribut entiteta** → **stupac** (column)

Modeliramo **stabilne** pojmove domene, ne UI forme.

Primarni i strani ključ

Primarni ključ (PK)

- jednoznačno identificira redak
- najčešće **surrogate key**: id integer/uuid
- prirodni ključ (OIB, registracijski broj) - oprez

Strani ključ (FK)

- povezuje dvije tablice
- referencijalni integritet: nema usera s club_id koji ne postoji
- određuje relaciju (1:N, M:N)

Prirodni podaci se mijenjaju ili nisu uvijek dostupni, id je stabilan. U našem projektu: Club 1 : 1 User, Club 1 : N Lifter.

Relacije: 1:1, 1:N, M:N (brzi pregled)

Relacija	Primjer iz projekta	Implementacija
1 : 1	Club $\leftrightarrow$ User	FK + UNIQUE; rijetko, često ista tablica
1 : N	Club $\rightarrow$ Lifter	FK na strani „N”; najčešća relacija
M : N	Lifter $\leftrightarrow$ Competition	pomoćna tablica (Registration)

Kad dođemo do prijave: Competition 1 : N Registration i Lifter 1 : N Registration.

Constrainti: što baza smije/ne smije dopustiti

Constraint	Što jamči	Primjer
NOT NULL	vrijednost mora postojati	name kluba
UNIQUE	nema duplikata	email, username
CHECK	vrijednost poštuje pravilo	težina > 0
FOREIGN KEY	relacija mora biti valjana	club_id postoji u clubs

Pravila u kodu nisu dovoljna.

Baza je autoritet.

Unique constraint: primjer s usernameom

- Username korisnika mora biti jedinstven
- Bez UNIQUE: login postaje dvosmislen ili nesiguran
- Unique constraint je zaštita od race conditiona:
 - dva requesta u isto vrijeme pokušaju kreirati isti username

Na obrani pitam: *gdje* se jedinstvenost provjerava - u kodu, u bazi ili oboje?

Normalizacija (minimalno što trebate znati)

- Cilj: smanjiti dupliciranje i anomalije (update/insert/delete)
- 1NF: atomarne vrijednosti (nema lista u stupcu)
- 2NF/3NF: atributi ovise o ključu, a ne o drugim atributima

Ako imate isti podatak na 3 mjesta, negdje će kad-tad postati netočan.

Tipične greške u modeliranju (iz prakse)

Greška	Posljedica
lista u jednom stupcu ("a,b,c")	nema query-ja po elementima
email kao primarni ključ	email se može promijeniti
bez constrainta „jer ćemo provjeriti u kodu”	race condition
denormalizacija bez razloga	neusklađenost podataka

Ispravan model = manje bugova i manje if-ova u kodu.

SQL je temelj (ORM je sloj iznad)

- ORM generira SQL (ili ga mapira)
- Ako ne razumijete SQL:
 - ne znate zašto je query spor
 - ne znate zašto relationship radi čudno
- Danas radimo ORM, ali s mentalnim modelom SQL-a

SQL: minimalni CRUD primjeri

```
-- CREATE
INSERT INTO clubs (name, city) VALUES ('Behemot', 'Zadar');

-- READ
SELECT id, name FROM clubs WHERE city = 'Zadar';

-- UPDATE
UPDATE clubs SET city = 'Biograd' WHERE id = 1;

-- DELETE
DELETE FROM clubs WHERE id = 1;
```

JOIN: zašto relacije postoje

```
SELECT u.id, u.username, c.name AS club_name  
FROM users u  
JOIN clubs c ON c.id = u.club_id  
WHERE u.role = 'club';
```

Relacije omogućuju query-je koji su prirodni i učinkoviti.

Index: kada i zašto?

- Index ubrzava WHERE, JOIN, ORDER BY
- Index košta:
 - zauzima prostor
 - usporava INSERT/UPDATE (mora se ažurirati)
- Unique constraint obično stvara index (npr. email)

Indeksiraj ono po čemu često filtriraš: email, club_id, competition_id.

Transakcije (što su i zašto?)

- Transakcija = skup DB operacija koje su **atomarne**
- ACID (kratko):
 - Atomicity: sve ili ništa
 - Consistency: constrainti vrijede
 - Isolation: konkurentni requestovi
 - Durability: zapis ostaje

Svaki request tipično ima svoju transakciju (commit/rollback).

Model vs Schema (DB vs API)

- **Model** (SQLAlchemy): kako se podatak sprema u bazu
- **Schema** (Pydantic): kako podatak putuje preko API-ja

Zašto razdvajamo?

- ne želimo izlagati sve DB stupce (npr. `password_hash`)
- različiti input/output oblici (create vs read)
- sigurnost i kontrola ugovora

SQLAlchemy 2.0: osnovni komadi

- **Engine:** konekcija prema DB-u
- **Session:** jedinica rada (unit of work)
- **Declarative model:** Python klase kao tablice
- **Query:** SELECT/INSERT/UPDATE/DELETE kroz session

Danas postavljamo engine + session i kreiramo prva dva modela.

Engine i session (async)

```
from sqlalchemy.ext.asyncio import (
    create_async_engine, async_sessionmaker, AsyncSession
)

engine = create_async_engine(DATABASE_URL, pool_pre_ping=True)
AsyncSessionLocal = async_sessionmaker(
    bind=engine, class_=AsyncSession,
    expire_on_commit=False
)

async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except:
            await session.rollback()
            raise
```

Async session per request, commit/rollback kontrolirano. Koristimo asyncpg driver. Connection string za async mora biti postgresql+asyncpg://..., a ne obični postgresql://. To je čest uzrok da projekt „radi na predavanju, a ne radi doma”.

Zašto session per request?

- Web aplikacija ima mnogo paralelnih requestova
- Jedna globalna session:
 - race condition
 - curenje stanja između requestova
 - čudni bugovi koje ne možeš reproducirati

U FastAPI dependencyju otvori session, na kraju je zatvori.

Model: Club (SQLAlchemy 2.0 style)

```
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import String

class Club(Base):
    __tablename__ = "clubs"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), unique=True, nullable=False)
    city: Mapped[str] = mapped_column(String(80), nullable=False)

    users: Mapped[list["User"]] = relationship(back_populates="club")
```

Model: User (role + club_id)

```
from sqlalchemy import String, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(20), nullable=False)

    club_id: Mapped[int | None] = mapped_column(ForeignKey("clubs.id"), nullable=True)
    club: Mapped["Club | None"] = relationship(back_populates="users")
```

Admin može imati club_id = NULL.

Enum ili string za role?

- String je jednostavan, ali riskira typo („admn”)
- Enum je sigurniji, ali traži malo više konfiguracije

Možemo krenuti sa stringom + validation u schemi, a kasnije prebaciti u Enum.

Naming conventions (zašto je bitno?)

- Tablice u množini (`users`, `clubs`)
- Foreign key: `club_id`
- Index/constraint imena: predvidljiva (pomaže migracijama)

Ako naming nije dosljedan, migracije i debug postaju teži.

Alembic: što je i zašto ga koristimo?

- Version control za DB shemu
- Svaka promjena modela → nova migracija
- Omogućuje:
 - timski rad bez „radi mi na mojoj bazi”
 - kontrolirane promjene u produkciji
 - rollback kad nešto pođe krivo

Alembic: osnovne naredbe

```
# inicijalizacija (jednom)
```

```
alembic init alembic
```

```
# generiranje migracije iz modela
```

```
alembic revision --autogenerate -m "init clubs and users"
```

```
# primjena migracija
```

```
alembic upgrade head
```

```
# rollback
```

```
alembic downgrade -1
```

Autogenerate nije magija (što provjeriti)

- Autogenerate uspoređuje **metadata** s bazom
- Ali može pogriješiti ili propustiti:
 - složene constraint-e
 - rename stupaca (može ispasti drop+create)

Uvijek pročitaj migraciju prije upgrade.

Kako izgleda migracija (primjer)

```
def upgrade() -> None:
    op.create_table(
        "clubs",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("name", sa.String(length=120), nullable=False),
        sa.Column("city", sa.String(length=80), nullable=False),
        sa.UniqueConstraint("name"),
    )
```

Gdje je UNIQUE i što sprječava?

Rollback (downgrade) i zašto ga trebate znati

- Kad migracija u produkciji pukne, rollback može spasiti situaciju
- U razvoju rollback pomaže za provjeru da migracije nisu „one-way”
- Ali rollback nije uvijek trivijalan - drop stupca znači nepovratan gubitak podataka

Ne oslanjaj se na rollback kao plan. Planiraj migracije.

Seed podaci: što, zašto, kako

- Seed = inicijalni podaci za razvoj/test (admin, demo klub)
- Nije isto što i migracija:
 - migracija mijenja shemu
 - seed puni tablice
- Seed mora biti **idempotentan** ili barem siguran za ponavljanje

Seed: minimalni cilj

U seed-u danas želimo:

- kreirati **admin** korisnika
- kreirati jedan **demo klub**
- kreirati jednog **club usera** za demo klub

Da možemo odmah testirati auth i ownership kada dođemo do JWT predavanja.

Timestampi i timezone (kratko, ali bitno)

- U produkciji:
 - spremamo vrijeme u UTC
 - prikaz radimo u lokalnoj zoni (UI)
- U našem projektu rokovi (prelim/final) će biti datetime:
 - backend uspoređuje s server vremenom (UTC)

Na obrani, ako datumi „ne rade”, pitam: timezone? format? gdje uspoređuješ?

Demo kroz commitove (Predavanje 2)

- `feat(api): add SQLAlchemy engine and session factory`
- `feat(api): add Club and User models with relationship`
- `chore(api): add Alembic and initial migration`
- `feat(api): create initial migration for Club and User`
- `feat(api): add seed script for admin and demo club`

Na satu: prolazimo diffeve i provjeravamo DB stanje.

Mini checklist: što mora raditi do kraja sata

- ❶ `docker compose up -d db` (Postgres radi)
- ❷ `alembic upgrade head` (tablice su kreirane)
- ❸ seed skripta se izvrši bez greške
- ❹ u bazi postoji admin user i demo klub

Ako ovo nemate, sve iduće (JWT, ownership, registracije) će vam biti teže.

Pitanja za diskusiju (za provjeru razumijevanja)

- Zašto je email UNIQUE? Što se dogodi bez toga?
- Zašto admin ima club_id = NULL?
- Što radi foreign key constraint?
- Što je razlika model vs schema?
- Zašto session per request, a ne globalna session?

Što ide sljedeće (Predavanje 3)

- Hashiranje lozinki
- JWT access + refresh token
- Login i refresh endpointi
- Dependency `get_current_user`
- Sigurnost u SPA (XSS i token storage)